PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

High Speed Link Lite

HBM 베이스 다이 디지털 링크 슬라이스 — 스펙 · 아키텍처 · RTL · 검증 · PD 가이드

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	용현 권
저장소	`github.com/yhkwon-git/High_Speed_Link_Lite` · commit `4feb39f` (2026-10-29)
사용 도구	SystemVerilog · Verilator 5.052 · Yosys 0.52 · Python (VCD · 커버리지 · 덱 자동화)

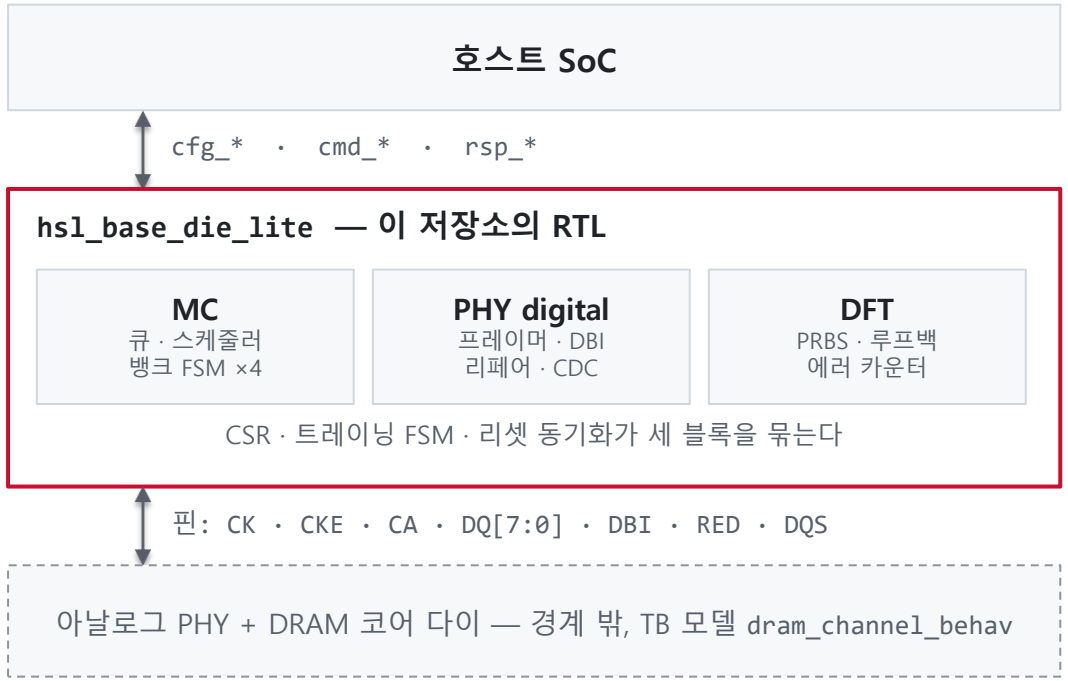
파형 · 커버리지 · 합성 수치는 모두 저장소 RTL/TB를 수정하지 않고 2026-10-30에 다시 실행해 얻은 결과다.

문제: 고속 링크에서 '디지털이 책임지는 부분'을 끝까지 닫기

HBM base die의 한 채널 슬라이스를 작은 파라미터로 줄여, 스펙부터 PD 가이드까지 혼자 검증할 수 있는 범위로 만든 스터디 설계

왜 이 프로젝트인가

- HBM base die는 호스트 SoC와 DRAM 코어 다이 사이에서 명령 스케줄링, 트레이닝, 데이터 인코딩, 테스트를 디지털로 중재한다.
- 고속 PHY는 아날로그 비중이 크다. 그래서 디지털이 책임질 **경계** — 핀의 논리값, 핸드셰이크, CDC, 리셋, DFT 관측점 — 를 문서와 RTL로 고정하는 것이 목표다.
- 실제 HBM 채널 폭 대신 DQ 8 · BL 4 · 뱅크 4로 줄였다. MC | PHY digital | DFT 구조와 설계 규칙은 그대로 유지한다.



범위 밖 — docs/01_requirements.md

멀티 채널 · 전력(mW) 사인오프 · 아날로그 마진 · JEDEC/DFI 호환 선언. 없는 숫자를 만들어 넣지 않는다.

3.2 Gb/s/DQ

목표 데이터율 · 컨트롤러 800 MHz / PHY 1.6 GHz · raw 3.2 GB/s

21 모듈 · 1,462줄

RTL 22파일 1,815줄 (주석·빈 줄 제외 1,462줄) · TB 1,108줄

13 / 13 PASS

SMOKE · T1–T8 · E7 · E9 · E11 · T7 unit을 한 번의 회귀로 (Verilator 5.052)

87.5% · 96.6%

라인 · 분기 커버리지, 이번 재실행에서 측정 (토글 56.6%)

스펙: 기능 7개와 숫자를 문서에 먼저 고정

숫자를 바꿀 때는 docs/01_requirements.md → hsl_pkg 순서. RTL에만 있는 숫자는 만들지 않는다.

기능 요구

1. 트레이닝이 끝나기 전에는 호스트 명령을 받지 않는다
2. 순서: RESET → CKE → MRS → ZQ → WRLVL → RDLVL → DESKEW → WRTRAIN → MISSION
3. MISSION 이후 ACT · PRE · RD · WR · REF (REF는 인코딩만 있고 발행하지 않음)
4. RD 응답은 버스트 하나, 준비 전에는 `rsp_valid` = 0
5. 루프백: TX 버스트를 RX로 되돌릴 수 있다
6. 레인 리페어: DQ 하나를 리던던트 핀(RED)으로 치환
7. ID · 버전 · 트레이닝 상태 · 에러 카운트를 읽을 수 있다

스터디 파라미터 (hsl_pkg)

항목	값
DQ / DBI / RED	8 / 1 / 1 → PIN_W 10
버스트	BL 4 beat (beat 0이 첫 UI)
뱅크 · Row · Col	4 · 8 bit · 4 bit
데이터 폭	32 bit = DQ_W × BL
목표 데이터율	3.2 Gb/s/DQ → raw 3.2 GB/s
클럭	ctrl 1.25 ns · PHY 0.625 ns

전력 · 신뢰성 규칙

- CKE는 FSM이 RESET일 때만 0
- DBI: beat의 1 개수 > DQ_W/2 이면 반전
- 드라이버 OE는 버스트 밖에서 0
- 에러 카운터는 16비트 포화, 랩어라운드 없음

타이밍 기본값 (ctrl 클럭)

파라미터	클럭	소유 블록
`tRCD`	4	bank FSM
`tRP`	4	bank FSM
`tRAS`	10	bank FSM
`tRTP`	3	bank FSM
`tWR`	4	bank FSM
`tCCD`	2	scheduler
`tWTR`	4	scheduler
`tRFC`	20	(REF 미발행)

수락 조건

- `[SMOKE] PASS`를 새 테스트가 깨지 않을 것
- `ready` = 0 구간의 명령 드롭은 실패
- 타임아웃은 실패, X를 데이터로 샘플하지 않음

아키텍처: 단일 클럭(A)으로 닫고, 2:1 PHY 클럭(B)으로 이행

세 후보를 성능 · 전력 · 면적 · 리스크로 비교 — UI 직렬화(C)는 아날로그 PHY 경계 밖에 둔다 (docs/02_architecture.md)

	A. 단일 클럭, 버스트 폭	B. 컨트롤러 1 : PHY 2	C. RTL에서 UI 직렬화
성능 스터디	뱅크 타이밍과 스케줄 효율	A + 기어박스 처리량	파형으로 eye 관찰이 쉬움
전력 스터디	DBI, CKE, OE	A와 같음, 클럭 하나 추가	핀 단위 토글이라 DBI 효과가 잘 보임
면적	32비트 데이터패스	16비트 PHY beat + 작은 FIFO	시프트 레지스터는 작지만 검증 시간이 김
리스크	CDC를 아직 다루지 않음	포인터 동기화 버그	시뮬이 느리고 아날로그처럼 보이기 쉬움
결정	**v0.1 출발점** — 핸드셰이크와 뱅크 타이밍을 먼저 닫음	**E7에서 이행** — `clk_phy`, 기어박스, 비동기 FIFO	미채택 — 핀 UI 직렬화는 아날로그 PHY 몫

A → B 이행 때 시뮬 클럭 · SDC · `hsl_pkg` 상수를 같은 식으로 다시 맞추고, 하나라도 다르면 미완료로 보는 대조표(E12)를 남겼다.

클럭 식을 문서에서 시작

CK = 3.2 Gb/s ÷ 2 = 1.6 GHz (625 ps), 컨트롤러 = CK ÷ 2 = 800 MHz. beats_per_burst = BL ÷ DDR_BITS = 2 → 컨트롤러 1클럭에 32b 버스트 1개

In-order 스케줄러

큐 head만 발행한다. 다른 뱅크가 열려 있어도 기다리므로 대역은 손해, 검증과 면적은 단순. 뱅크 FSM을 밖에 둬서 OoO로 바꿔도 큐 구조는 유지

Beat-major 데이터 배치

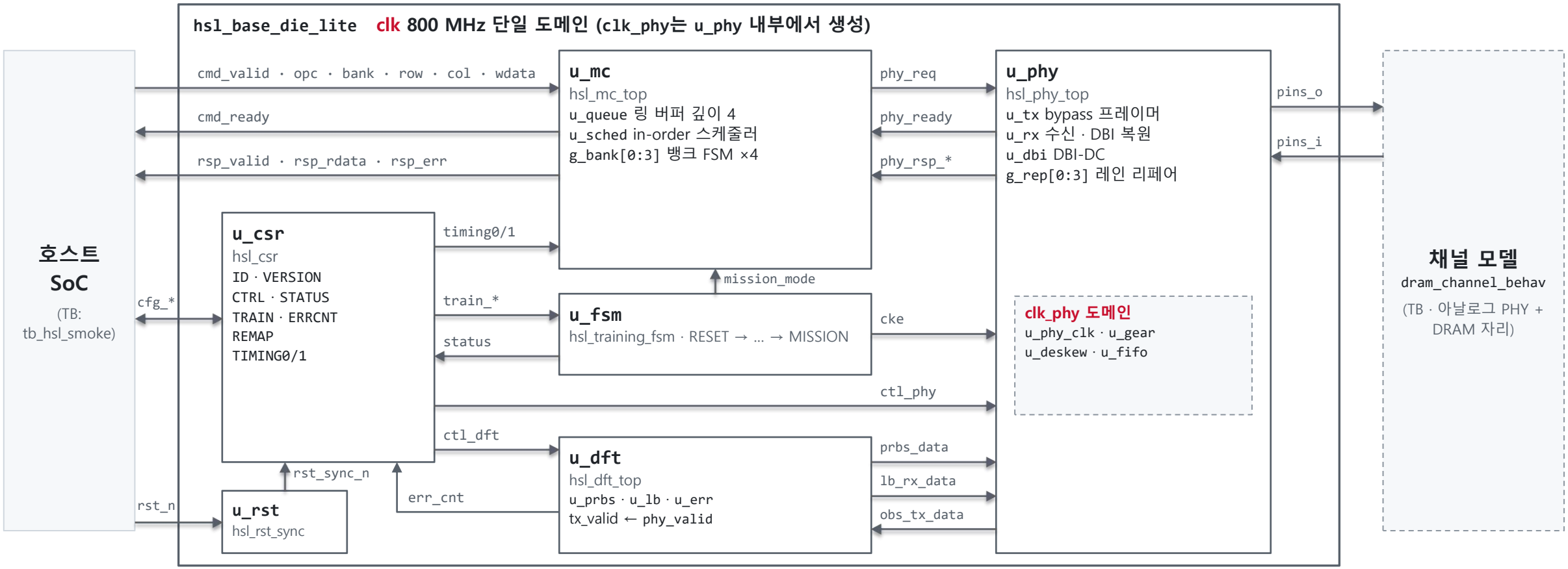
beat b = `data[b*DQ_W +: DQ_W]`. DBI · ECC가 한 UI의 바이트를 한 덩어리로 다룬다. 레인 단위로 저장하면 비트를 흩어 읽어야 함

리셋 도메인 분리

CSR은 `rst_sync_n`만, 데이터패스는 `rst_sync_n & ~soft_rst`. CSR까지 넣으면 soft_rst 비트가 스스로 지워져 리셋을 유지할 수 없음

Top level: 6개 인스턴스, 하나의 리셋 트리, 하나의 컨트롤러 클럭

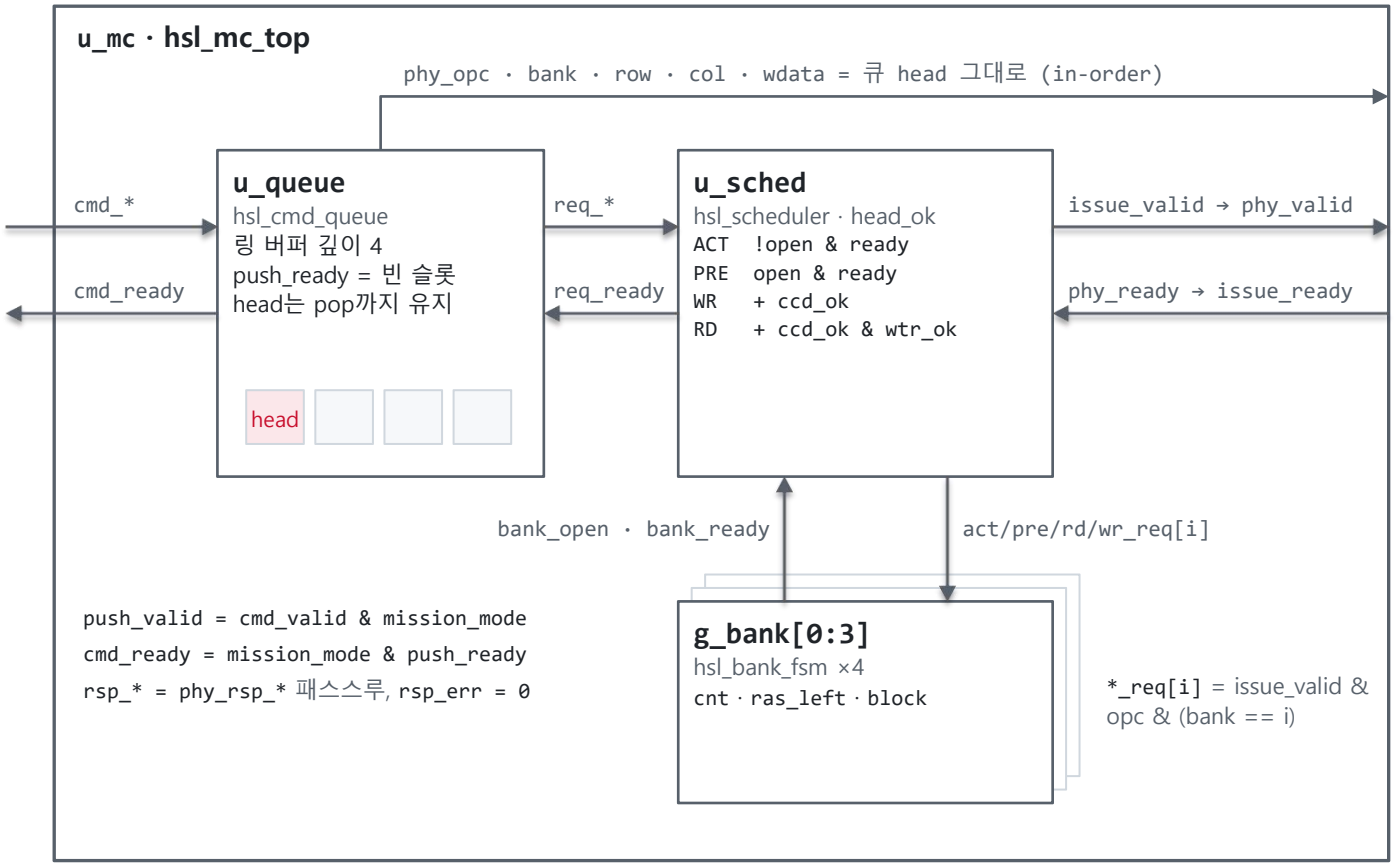
rtl/top/hsl_base_die_lite.sv의 인스턴스 이름과 포트 이름 그대로 — 편집 가능한 도형



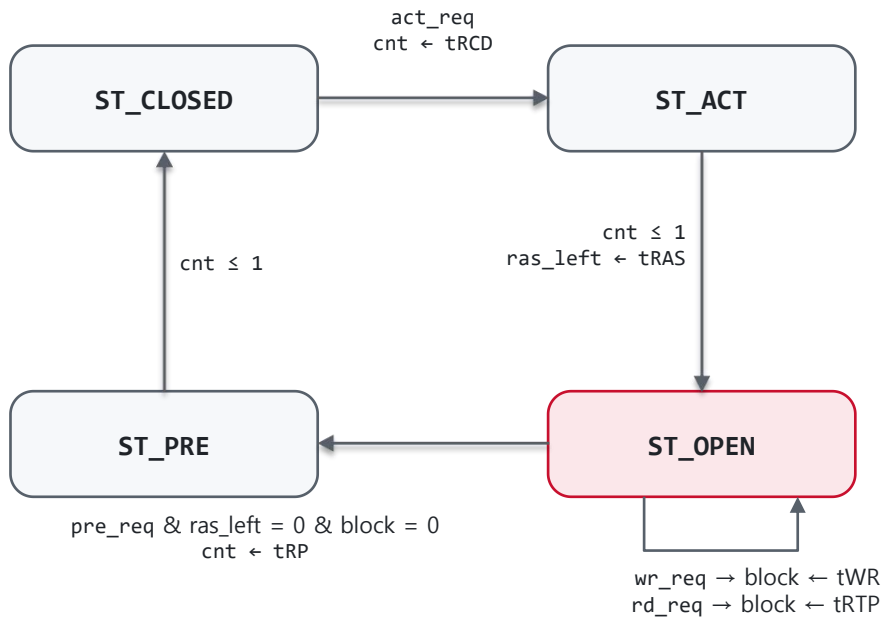
범례 `phy_req` = phy_valid, phy_opc/bank/row/col/wdata · `status` = train_state, train_done, mission_mode · `train_*` = train_bypass/start/wait, zq/wrlvl/rdlvl/deskew_done
`ctl_phy` = dbi_en, phy_bypass, repair_en/lane, loopback, prbs_en · `ctl_dft` = loopback, prbs_en, err_clear · `pins_o` = pin_ck/cke/ca/dq_o[9:0]/dq_oe/dqs_o · `pins_i` = pin_dq_i[9:0], pin_dqs_i
리셋: `u_csr`는 `rst_sync_n`, 나머지 네 블록은 `rst_logic_n = rst_sync_n & ~soft_rst` (top의 assign 한 줄)

MC: 큐 → in-order 스케줄러 → 뱅크 FSM ×4, 타이밍 소유를 나눔

tRCD · tRP · tRAS · tWR · tRTP는 뱅크 FSM, tCCD · tWTR는 스케줄러가 소유 (rtl/mc/*.sv)



hsl_bank_fsm 상태도 (뱅크마다 1개)

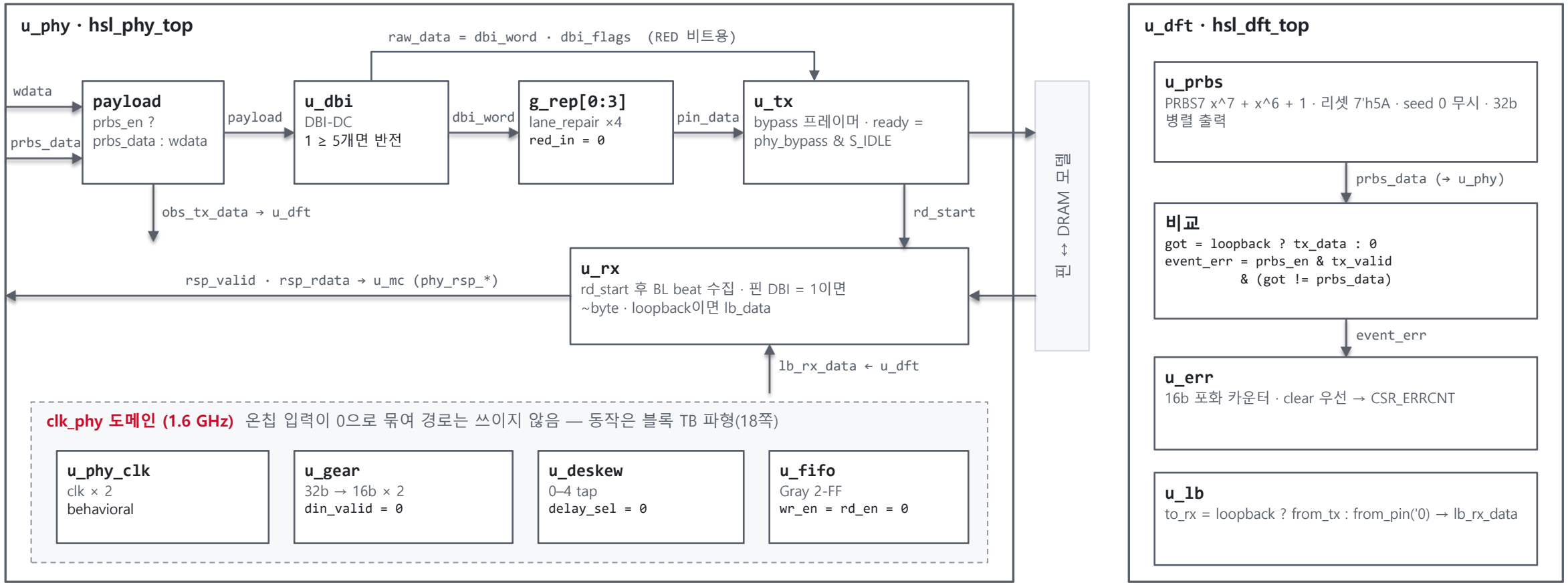



```
timing_ok = ST_CLOSED | (ST_OPEN & ras_left == 0 & block == 0)
is_open   = ST_OPEN
```

관찰 컬럼 명령(RD/WR)도 `ras_left = 0`까지 기다린다 → T4에서 ACT → WR 발행 간격 15클럭. tRCD 뒤 컬럼을 허용하는 JEDEC보다 보수적인 스터디 규칙이다.

PHY · DFT: bypass 프레이머가 핀을 몰고, clk_phy 블록은 따로 검증

hsl_phy_top.sv — 온칩 gearbox · FIFO · deskew는 입력이 0으로 묶여 있어 tb_hsl_blocks의 독립 인스턴스로 확인



`tx_valid = phy_valid` (MC가 PHY로 내는 명령 박자마다 비교) · 핀 루프백은 이 비교에 들어가지 않는다 (docs/06_dft_plan.md) · RX 쪽은 RED 핀을 쓰지 않아 리페어는 TX 핀 레벨까지만 검증

리셋 트리와 트레이닝 FSM: 비동기 assert · 동기 deassert, done 펄스 대기

핀 rst_n → 2-FF 동기화 → CSR은 rst_sync_n, 나머지는 rst_logic_n · FSM은 done 펄스가 오기 전에는 상태를 유지

rtl/top/hsl_rst_sync.sv L19–27

```
19  always_ff @(posedge clk or negedge rst_ni) begin
20    if (!rst_ni) begin
21      sync_r <= 2'b00;
22    end else begin
23      sync_r <= {sync_r[0], 1'b1};   ①
24    end
25  end
26
27  assign rst_no = sync_r[1];
```

rtl/top/hsl_base_die_lite.sv L73, L75–77, L105–107

```
73  assign rst_logic_n = rst_sync_n & ~soft_rst;   ②
    ----------------------------------------------------
75  hsl_csr u_csr (
76    .clk          (clk),
77    .rst_n        (rst_sync_n),   ③
    ----------------------------------------------------
105 hsl_training_fsm u_fsm (
106   .clk          (clk),
107   .rst_n        (rst_logic_n),
```

rtl/ctrl/hsl_training_fsm.sv L38–40, L42–57

```
38  assign train_done   = (state == TR_MISSION);
39  assign mission_mode = train_done;
40  assign cke          = (state != TR_RESET);   ④
    ----------------------------------------------------
42  always_ff @(posedge clk or negedge rst_n) begin
43    if (!rst_n) begin
44      state <= TR_RESET;
45    end else if (train_bypass) begin   ⑤
46      state <= TR_MISSION;
47    end else begin
48      case (state)
49        TR_RESET:   if (train_start) state <= TR_CKE;
50        TR_CKE:     state <= TR_MRS;
51        TR_MRS:     state <= TR_ZQCAL;
52        TR_ZQCAL:   if (zq_done) state <= TR_WRLVL;   ⑥
53        TR_WRLVL:   if (wrlvl_done) state <= TR_RDLVL;
54        TR_RDLVL:   if (rdlvl_done) state <= TR_DESKEW;
55        TR_DESKEW:  if (deskew_done) state <= TR_WRTRAIN;
56        TR_WRTRAIN: state <= TR_MISSION;
57        TR_MISSION: state <= TR_MISSION;
```

1. `sync_r <= {sync_r[0], 1'b1}` — assert는 비동기, 해제는 clk 상승 2번 뒤. 원시 핀은 다른 블록이 직접 쓰지 않는다
2. `rst_logic_n = rst_sync_n & ~soft_rst` — soft reset은 MC · FSM · PHY · DFT만 지운다
3. CSR은 `rst_sync_n`만 받는다 → CTRL.soft_rst 비트가 자기 자신을 지우지 않아 리셋을 유지할 수 있다
4. `cke = (state != TR_RESET)` — RESET에서만 CKE 0. 스모크가 핀으로 확인
5. `train_bypass`가 case보다 우선 → 1클럭 만에 MISSION. T4–T8이 이 경로로 빠르게 진입
6. ZQCAL · WRLVL · RDLVL · DESKEW는 done 펄스가 올 때까지 머문다 (T1)

검증 연결 SMOKE — 리셋 직후 RESET · CKE 0 · cmd_ready 0, bypass 후 MISSION, soft_rst 동안 RESET 후 복귀 (expect_eq 27곳) · T1 — done 펄스 대기와 상태 순서 · 파형은 14 · 15쪽

스케줄러는 head만 보고, 타이밍은 뱅크 FSM이 판정한다

명령 드롭이 없도록 req_ready를 head_ok와 PHY ready로만 만든다 — 뱅크 타이밍은 timing_ok 한 비트로 요약

rtl/mc/hsl_scheduler.sv L35–38, L43–51, L56–57

```
35  wire ccd_ok = (ccd_cnt == 8'd0);
36  wire wtr_ok = (wtr_cnt == 8'd0);
37
38  wire act_ok = !bank_open[req_bank] && bank_ready[req_bank];   (1)
    ----------------------------------------------------------------
43    head_ok = 1'b0;
44    if (req_valid) begin
45      case (req_opc)
46        OPC_NOP: head_ok = 1'b1;
47        OPC_ACT: head_ok = act_ok;
48        OPC_PRE: head_ok = col_ok;
49        OPC_RD:  head_ok = col_ok && ccd_ok && wtr_ok;          (2)
50        OPC_WR:  head_ok = col_ok && ccd_ok;
51        default: head_ok = 1'b0;                                (3)
    ----------------------------------------------------------------
56  assign req_ready   = head_ok && issue_ready;                  (4)
57  assign issue_valid = req_valid && req_ready;
```

rtl/mc/hsl_bank_fsm.sv L48–50, L75–87

```
48  assign is_open   = (st == ST_OPEN);
49  assign timing_ok = (st == ST_CLOSED) ||
50                     ((st == ST_OPEN) && (ras_left == 8'd0) && (block == 8'd0));   (5)
    ----------------------------------------------------------------
75        ST_OPEN: begin
76          if (ras_left != 8'd0) begin                           (6)
77            ras_left <= ras_left - 8'd1;
78          end else if (block != 8'd0) begin
79            block <= block - 8'd1;
80          end else if (pre_req) begin
81            st  <= ST_PRE;
82            cnt <= t_rp_u;
83          end else if (wr_req) begin                            (7)
84            block <= t_wr;
85          end else if (rd_req) begin
86            block <= t_rtp;
87          end
```

1. act_ok · col_ok: 뱅크의 is_open · timing_ok만 본다 (뱅크 타이밍은 뱅크 FSM 소유)
2. RD는 tCCD와 tWTR, WR는 tCCD를 추가로 확인 (스케줄러 소유 카운터)
3. REF · 예약 opcode는 default → head_ok = 0. head에 오면 큐가 멈춤 (19쪽 D)
4. req_ready = head_ok && issue_ready — PHY가 바쁘면 pop하지 않아 드롭이 없다
5. timing_ok: CLOSED이거나, OPEN이면서 ras_left와 block이 모두 0
6. OPEN에서는 tRAS 카운트다운이 먼저 — 컬럼 명령도 tRAS 뒤 (T4: ACT → WR 15클럭)
7. WR는 block ← tWR, RD는 block ← tRTP — 다음 PRE와 컬럼 명령을 막는다

CDC: Gray 포인터 비동기 FIFO와 토글 핸드셰이크 기어박스

쓰기 clk_phy → 읽기 clk · 포인터는 Gray로 2-FF 동기화, 기어박스는 토글 한 쌍이 32b 워드 하나

rtl/phy/hsl_cdc_fifo.sv L40-42, L44-47, L65, L70-75

```
40  function automatic logic [AW:0] bin2gray(input logic [AW:0] b);
41    bin2gray = (b >> 1) ^ b;                                          ①
42  endfunction

44  assign wgray    = bin2gray(wbin);
45  assign rgray    = bin2gray(rbin);
46  assign wr_full  = (wgray == {~rptr_g_s2[AW], ~rptr_g_s2[AW-1], rptr_g_s2[AW-2:0]}); ②
47  assign rd_empty = (rgray == wptr_g_s2);

65  always_ff @(posedge clk_r or negedge rst_n) begin

70    end else begin
71      wptr_g_s1 <= wgray;                                             ③
72      wptr_g_s2 <= wptr_g_s1;
73      if (rd_en && !rd_empty) begin
74        rbin <= rbin + 1'b1;
75      end
```

① bin2gray: 포인터를 Gray로 넘겨 한 번에 1비트만 바뀜 → 동기화 중 잘못 샘플해도 인접 값

② full은 상위 2비트를 뒤집어 비교, empty는 동기화된 쓰기 포인터와 같을 때

③ 읽기 클럭의 `wptr_g_s1 → s2` 2-FF. SDC는 false path 대신 `set_max_delay 1.25`

rtl/phy/hsl_gearbox.sv L35, L46-48, L63-68, L74-78

```
35  assign din_ready = (req_tog == ack_s2);                             ④

46    if (din_valid && din_ready) begin
47      hold    <= din_ctrl;
48      req_tog <= ~req_tog;

63    req_s1 <= req_tog;
64    req_s2 <= req_s1;
65    if (!beat) begin
66      if (req_s2 != ack_tog) begin                                    ⑤
67        shift      <= hold;
68        dout_phy   <= hold[DQ_W*DDR_BITS-1:0];

74    end else begin
75      dout_phy   <= shift[DATA_W-1:DQ_W*DDR_BITS];
76      dout_valid <= 1'b1;
77      beat       <= 1'b0;
78      ack_tog    <= ~ack_tog;                                         ⑥
```

④ `din_ready = (req_tog == ack_s2)` — 토글 한 쌍이 워드 하나

⑤ PHY 클럭에서 `req_s2 != ack_tog`이면 하위 16b부터 내보냄

⑥ 두 번째 beat 뒤 `ack_tog` 토글 → ctrl 쪽 2-FF 후 다음 워드

인코딩과 DFD: DBI-DC, 리던던트 레인, 포화 에러 카운터

핀 high 수를 줄이는 DBI, 배드 레인 하나를 RED로 우회하는 리페어, 고장을 숨기지 않는 16b 카운터

rtl/phy/hsl_dbi.sv L17–33

```
17  function automatic logic [3:0] pop8(input logic [7:0] b);
18    pop8 = {3'b0, b[0]} + b[1] + b[2] + b[3] + b[4] + b[5] + b[6] + b[7];  (1)
19  endfunction
20
21  integer bi;                                                              (2)
22  always_comb begin
23    dout = din;
24    dbi  = '0;
25    if (en) begin
26      for (bi = 0; bi < BL; bi = bi + 1) begin
27        if (pop8(din[bi*DQ_W +: DQ_W]) > 4'd4) begin                       (3)
28          dout[bi*DQ_W +: DQ_W] = ~din[bi*DQ_W +: DQ_W];
29          dbi[bi] = 1'b1;
30        end
31      end
32    end
33  end
```

(1) pop8: beat 하나의 1 개수 (4비트 합)

(2) 모듈 스코프 `integer bi` — Verilator는 통과, Yosys 0.52는 latch로 거부 (20쪽)

(3) 1이 4개보다 많으면 beat 반전 + 플래그 → 핀 high 수 ≤ DQ_W/2

rtl/phy/hsl_phy_tx.sv L63–68, L136–138

```
63  function automatic logic red_of(input logic [2:0] beat);
64    if (repair_en && (repair_lane < 4'd8)) begin
65      red_of = raw_r[beat*DQ_W + 32'(repair_lane)];                        (4)
66    end else begin
67      red_of = 1'b0;
68    end
    ----------------------------------------------------------------------
136       S_WD: begin
137         ca    <= {2'b00, bank_r, OPC_WR};
138         dq_o  <= {red_of(idx), dbi_r[idx[1:0]], data_r[idx*DQ_W +: DQ_W]};  (5)
```

(4) `red_of`: 리페어 레인의 원래 비트(DBI 적용 후 `raw_r`), lane ≥ 8이면 0

(5) 데이터 beat 핀 = {RED, DBI, DQ[7:0]}. DQ[lane]은 `g_rep`가 0으로 치환

rtl/dft/hsl_err_cnt.sv L22–25

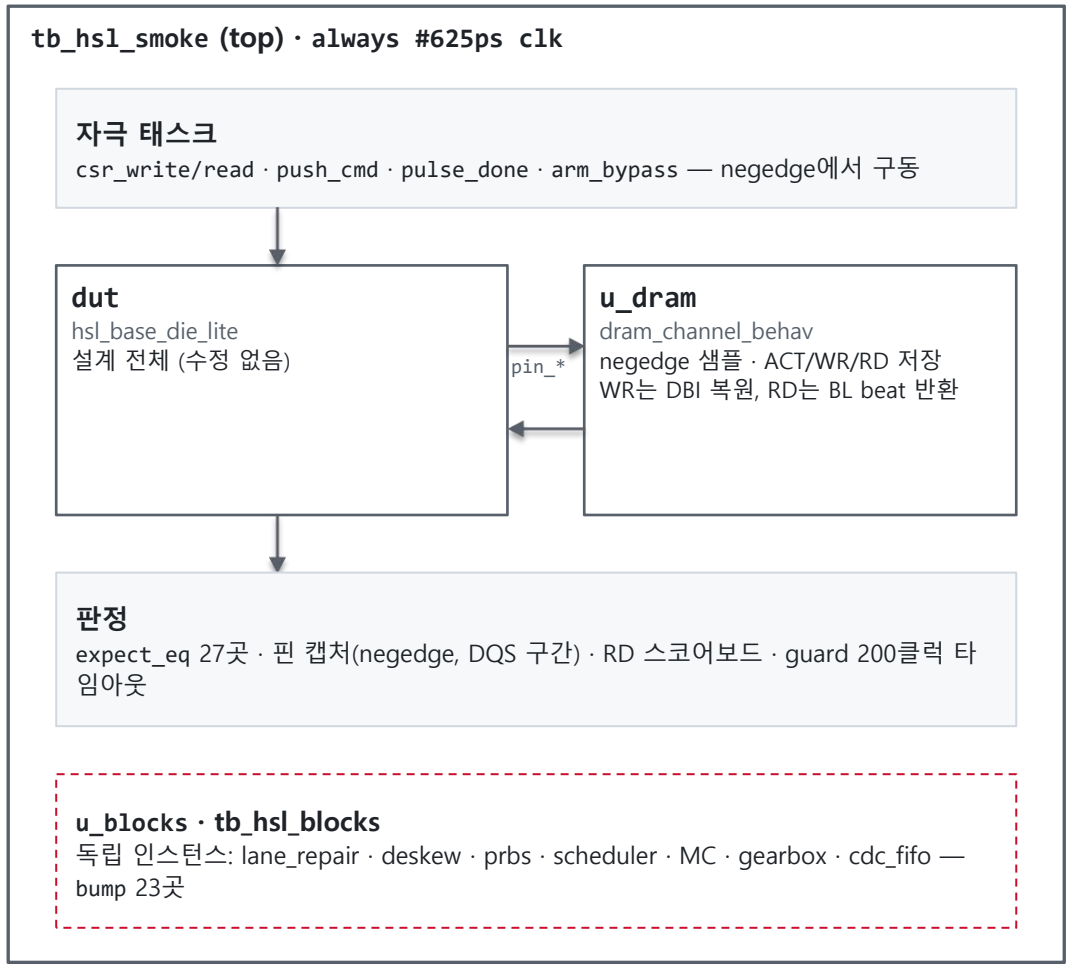
```
22    end else if (clear) begin                                              (6)
23      count <= '0;
24    end else if (event_valid && event_err && (count != 16'hFFFF)) begin    (7)
25      count <= count + 16'd1;
```

(6) `clear`가 같은 클럭의 에러보다 우선

(7) `16'hFFFF`에서 멈춤 — 고장이 계속될 때 0으로 돌아가 정상처럼 보이지 않게

검증: 스모크 · 블록 · 채널 3계층을 한 번의 회귀로

tb_hsl_smoke가 tb_hsl_blocks를 함께 인스턴스 — 한 프로세스에서 모든 테스트가 돌고, [ALL] PASS가 아니면 0이 아닌 종료 코드



ID	대상	확인 내용	위치
SMOKE	기본 동작	ID · VERSION · TIMING 리셋값, 미정의 주소 0, 리셋 직후 핀 · ready 0, bypass → MISSION, 깊이 4 full → ready 0, soft_rst	smoke
T1	트레이닝	순서대로 진행, done 펄스 전에는 정지	smoke
T2	뱅크 FSM	tRCD 전 RD 거절, tRP 전 ACT 거절	blocks
T3	스케줄러	tCCD 전 다음 컬럼 명령 거절	blocks
T4	채널	ACT → WR → RD, A5A5_5A5A 일치	smoke
T5	DBI	0x1F → 핀 E0 + DBI 1, 0x0F → 그대로	smoke
T6	CDC FIFO	두 클럭에서 3워드 순서 유지	blocks
T7	리페어	unit: lane 3 치환 · lane 8 무시 · en 0 / 핀: F7 + RED 1, FF	둘 다
T8	PRBS	루프백 일치 → ERRCNT 0, 불일치 → 증가	smoke
E7	기어박스	32b → C3D4, A1B2 두 beat	blocks
E9	deskew	tap 0 통과, tap 2 지연	blocks
E11	PRBS	리셋 시드 5A 첫 비트 1, seed 0 load 무시	blocks

통과 규칙: 이름 있는 FAIL과 0이 아닌 종료 코드 · 타임아웃은 실패 · X를 데이터로 샘플하지 않음 · 구동하지 않는 핀은 0 (docs/04_verification_plan.md)

결과: 13개 테스트 PASS · 라인 87.5% · 분기 96.6% · 토글 56.6%

이번 재실행에서 저장소 RTL/TB를 수정 없이 Verilator 5.052로 다시 실행해 측정 — 빌드 4.2 s, 시뮬 시간 511 ns

sim/regress.log

```
[T7] unit PASS
[SMOKE] PASS
[E9] PASS
[T1] PASS
[E11] prbs PASS
[T4] PASS
[T3] PASS
[T5] PASS
[T7] PASS
[T8] PASS
[T2] PASS
[E7] PASS
[T6] PASS
[ALL] PASS
```

재현: `sim/run_smoke.sh`와 같은 Verilator 명령 (`--binary --timing`). 커버리지는 `--coverage-line --coverage-toggle` 별도 빌드, 파형은 `--trace` 래퍼로 수집

모듈	라인	분기	토글
training_fsm	92.9%	100.0%	75.0%
csr	80.0%	100.0%	33.7%
rst_sync	100.0%	100.0%	60.0%
cmd_queue	100.0%	100.0%	45.1%
scheduler	90.0%	83.3%	29.5%
bank_fsm	90.0%	100.0%	21.3%
phy_tx	83.3%	100.0%	81.8%
phy_rx	66.7%	87.5%	60.8%
dbi	100.0%	100.0%	95.7%
lane_repair	100.0%	100.0%	93.8%
gearbox	100.0%	100.0%	51.6%
deskew	100.0%	75.0%	93.0%
cdc_fifo	100.0%	100.0%	31.0%
prbs	87.5%	100.0%	89.2%
err_cnt	66.7%	100.0%	21.4%
합계	87.5%	96.6%	56.6%

음영 = 놓친 라인/분기가 있는 모듈 · 모듈별로 인스턴스 합집합 (블록 TB 포함) · 합계에는 표에 없는
top · mc_top · dft_top · loopback · phy_top · phy_clk도 포함

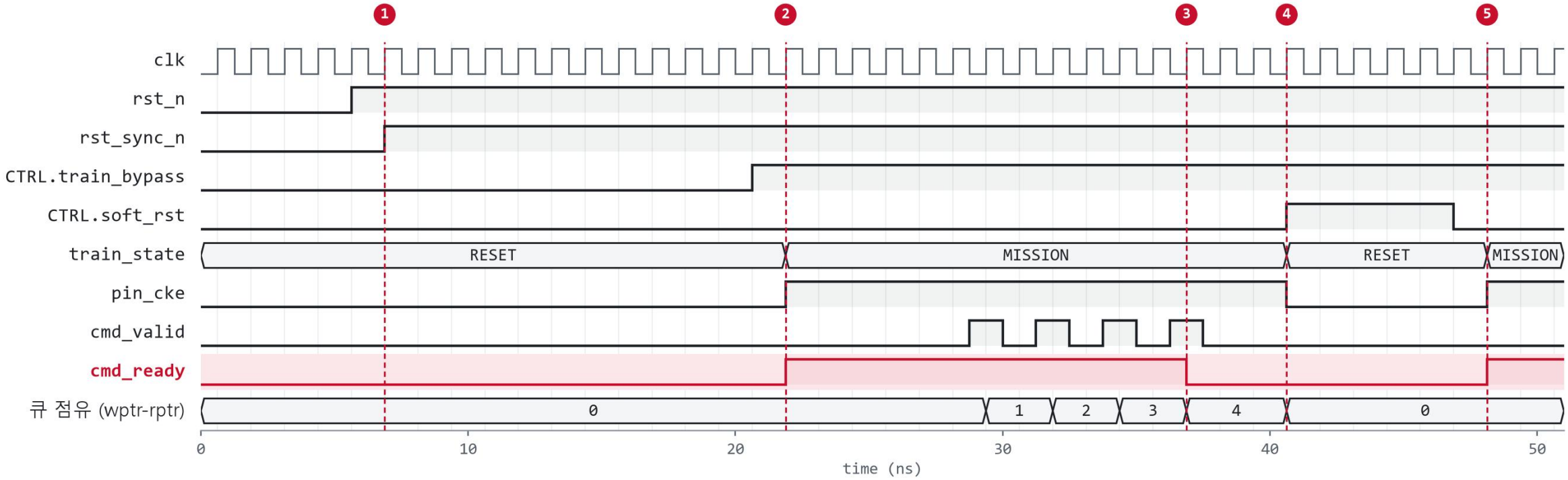
커버리지 구멍 → 다음 테스트

- `csr` ERRCNT · TIMING0/1 쓰기 미실행 → 타이밍 재프로그램, ERRCNT clear 테스트
- `err_cnt` clear 경로 미실행 → 같은 클럭의 clear 대 에러 우선순위
- `prbs` 0이 아닌 seed load 미실행 (온칩 load = 0)
- `scheduler` REF · 예약 opcode, tCCD · tWTR ≤ 1 분기 미도달
- `phy_rx` 핀 DBI = 1 수신 · 루프백 RX 응답(rd_start & loopback) 미실행 → DBI · 루프백 읽기 테스트
- `phy_tx` PRE 핀 프레임, `deskew` delay_sel > 4 클램프 미실행
- 토글 56.6%: 데이터 · 주소 상위 비트가 거의 고정 → 랜덤 패턴 필요

기능 커버리지(covergroup)는 저장소에 없음

리셋 해제 → bypass MISSION → 큐 full 백프레셔 → soft reset 복귀

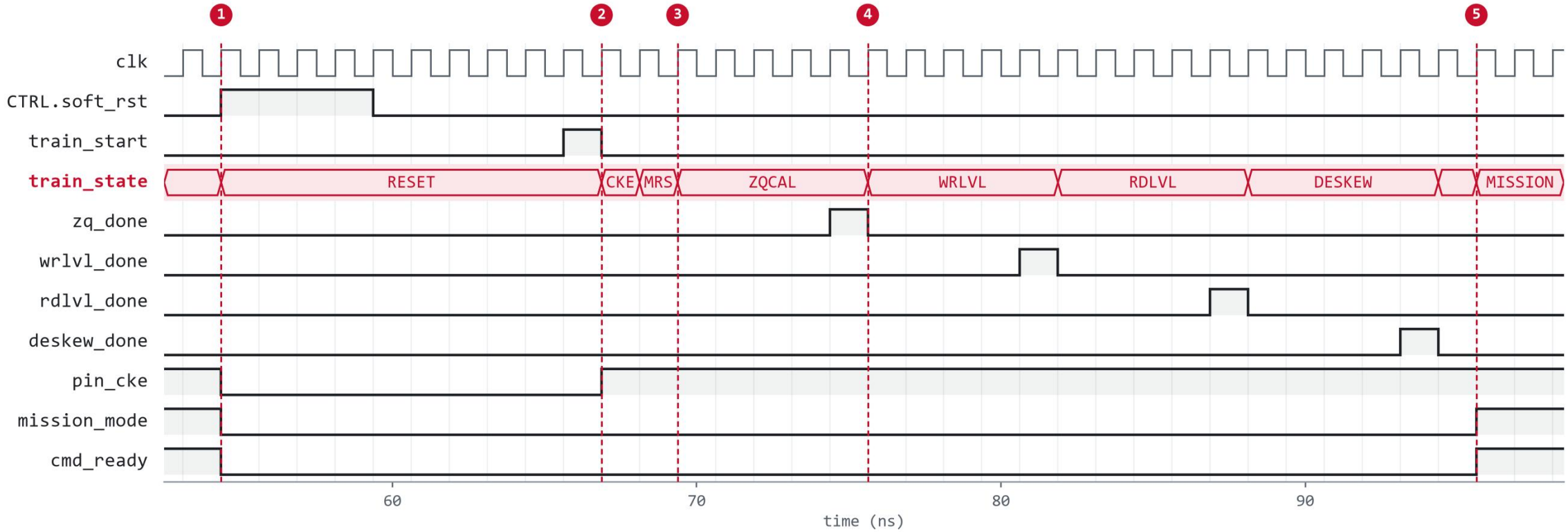
tb_hsl_smoke 0–51 ns · Verilator 5.052 --trace VCD를 파싱해 그린 실제 파형 · 신호 이름은 RTL 계층 그대로



1. 6.9 ns 핀 `rst_n` 해제 뒤 2-FF 동기화 → `rst_sync_n` 1 (해제가 clk 엣지와 겹침, 19쪽 B)
2. 21.9 ns `train_bypass` 쓰기 → 다음 클럭 MISSION · `pin_cke` 1 · `cmd_ready` 1
3. 36.9 ns push 4번으로 점유 4 → `cmd_ready` 0. `phy_bypass` = 0이라 PHY ready 0, pop 없음
4. 40.6 ns `soft_rst` → RESET · CKE 0 · 큐 비움. CSR은 유지 (bypass 1 그대로)
5. 48.1 ns `soft_rst` 해제 → bypass가 남아 있어 곧바로 MISSION, `cmd_ready` 1

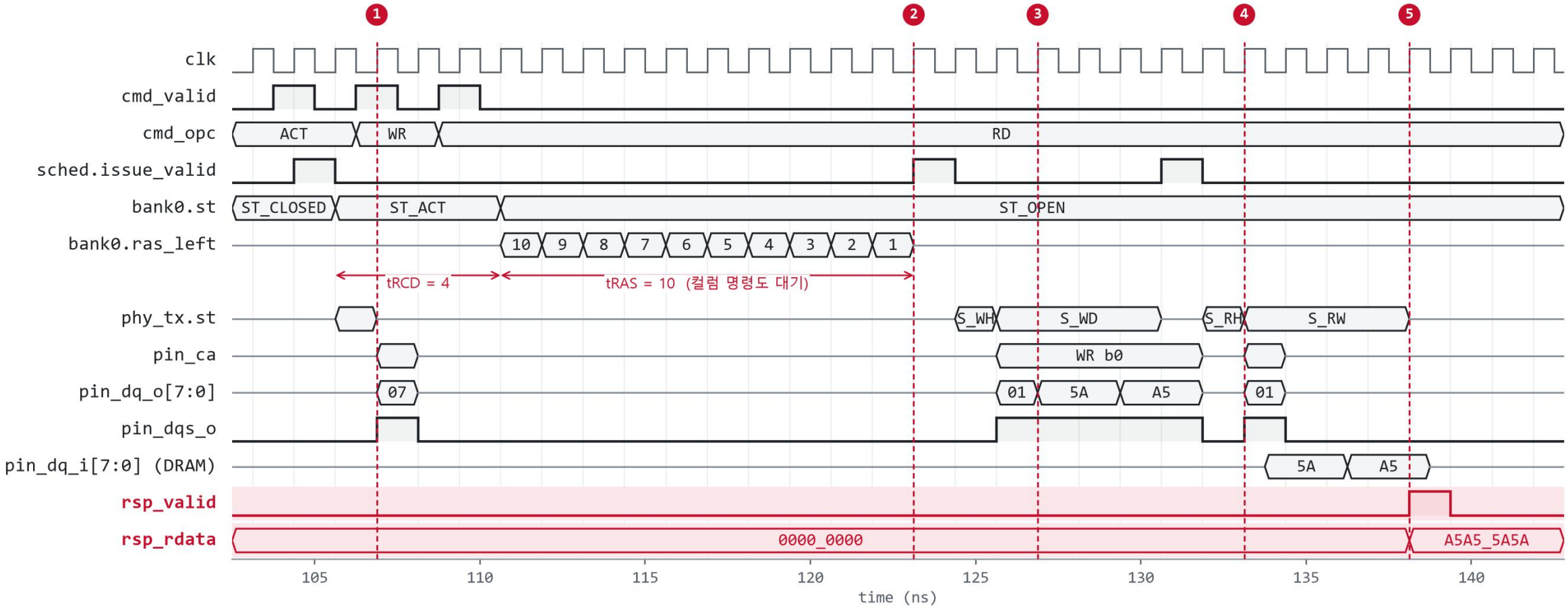
트레이닝: done 펄스마다 한 상태씩 진행, MISSION 전에는 명령 차단

tb_hsl_smoke T1 구간 · TB는 각 대기 상태에서 3클럭 이상 머무는지 본 뒤 done을 1클럭 펄스로 준다



1. 54.4 ns CTRL = soft_rst만 → RESET, bypass 0. T1 시작
2. 66.9 ns `train_start` 펄스 → CKE 상태, `pin_cke` 1 → MRS 1클럭
3. 69.4 ns ZQCAL은 `zq_done`까지 머묾. done 전에 WRLVL로 가면 T1 실패
4. 75.6 ns done 펄스 하나에 한 상태씩 — WRLVL → RDLVL → DESKEW
5. 95.6 ns WRTRAIN 1클럭(라벨 생략) 뒤 MISSION → `mission_mode` · `cmd_ready` 1

T4: ACT → WR → RD가 핀 프레임으로 나가고, 읽은 값 A5A5_5A5A가 일치

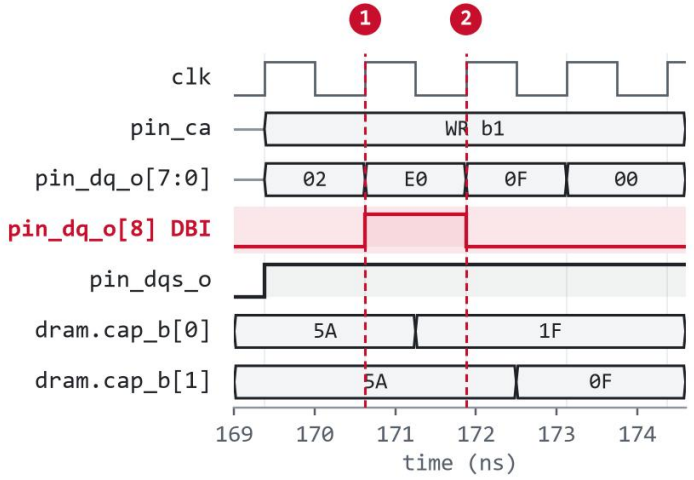


1. 106.9 ns ACT 핀 프레임: `pin_ca` ACT, DQ에 row 07. bank0 ST_ACT, tRCD 4
2. 123.1 ns `ras_left` 0 뒤에야 WR 발행 — ACT 발행부터 15클럭
3. 126.9 ns WR 헤더(col 01) 뒤 beat 5A 5A A5 A5 (같은 값 연속은 한 구간)
4. 133.1 ns PHY가 S_IDLE로 돌아온 뒤 RD 발행 → RD 헤더, DRAM 모델이 `pin_dq_i` 구동
5. 138.1 ns `rsp_rdata` = A5A5_5A5A 일치. RD 발행 → 응답 6클럭

핀에서 보이는 DBI 반전, RED 치환, PRBS 에러 카운트

tb_hsl_smoke의 같은 VCD에서 테스트별 구간을 잘라 그림 · 버스 값은 16진수

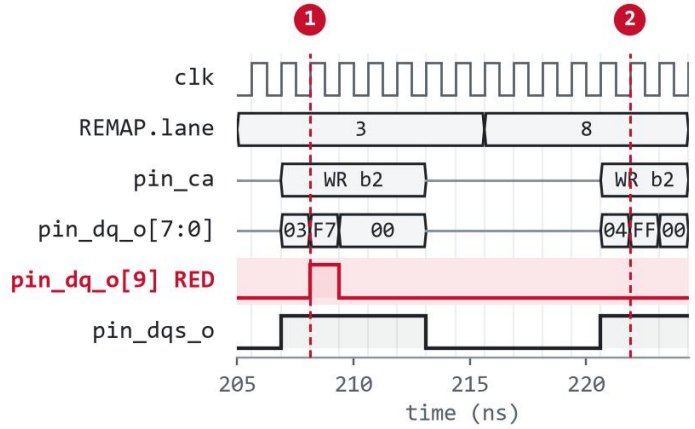
T5 · DBI-DC



1. 170.6 ns beat0 `0x1F` (1이 5개) → 핀 `E0` + DBI 1
2. 171.9 ns beat1 `0x0F` (4개) → 그대로, DBI 0

DRAM 모델이 DBI를 풀어 cap_b = 1F · 0F로 저장. beat0의 high 핀 5 → 4 (E0의 3 + DBI)

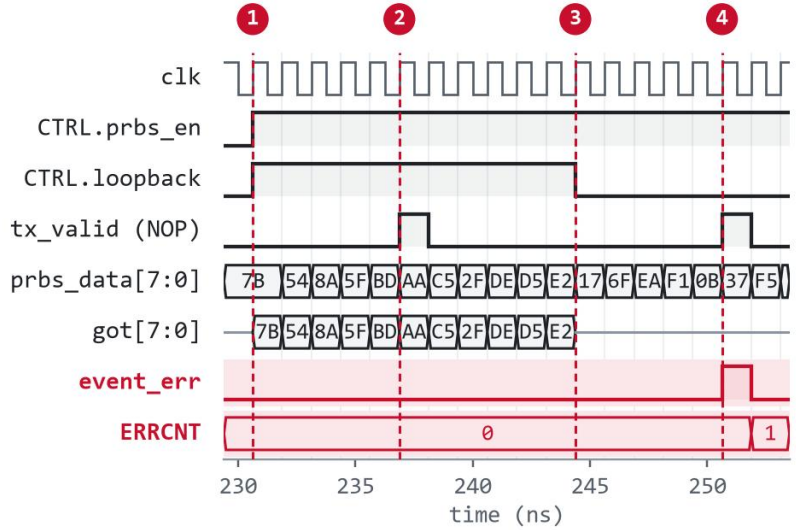
T7 · 레인 리페어



1. 208.1 ns REMAP lane 3 → 핀 `F7` (bit3 → 0), RED = 원래 bit3 = 1
2. 221.9 ns lane 8 → 치환 없음, 핀 `FF` · RED 0

TX 핀 레벨까지만 확인 — DRAM 모델과 u_rx는 RED를 쓰지 않아 복원 경로는 검증하지 않음

T8 · PRBS 병렬 루프백



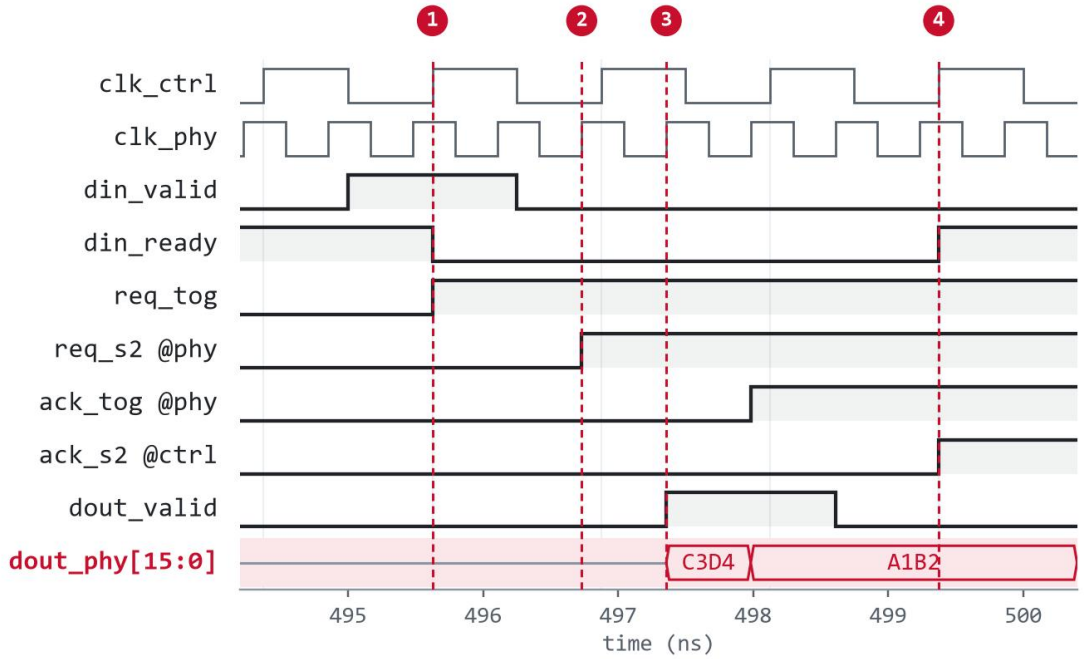
1. 230.6 ns `prbs_en` · `loopback` → `got` = `prbs_data`
2. 236.9 ns NOP 박자(`tx_valid`)에서 일치 → `event_err` 0
3. 244.4 ns `loopback` 0 → `got` = 0
4. 250.6 ns 불일치 → `event_err` → ERRCNT 1

표시는 하위 8비트, 비교는 32비트 전체

CDC 블록: 토글 핸드셰이크 기어박스와 Gray 포인터 비동기 FIFO

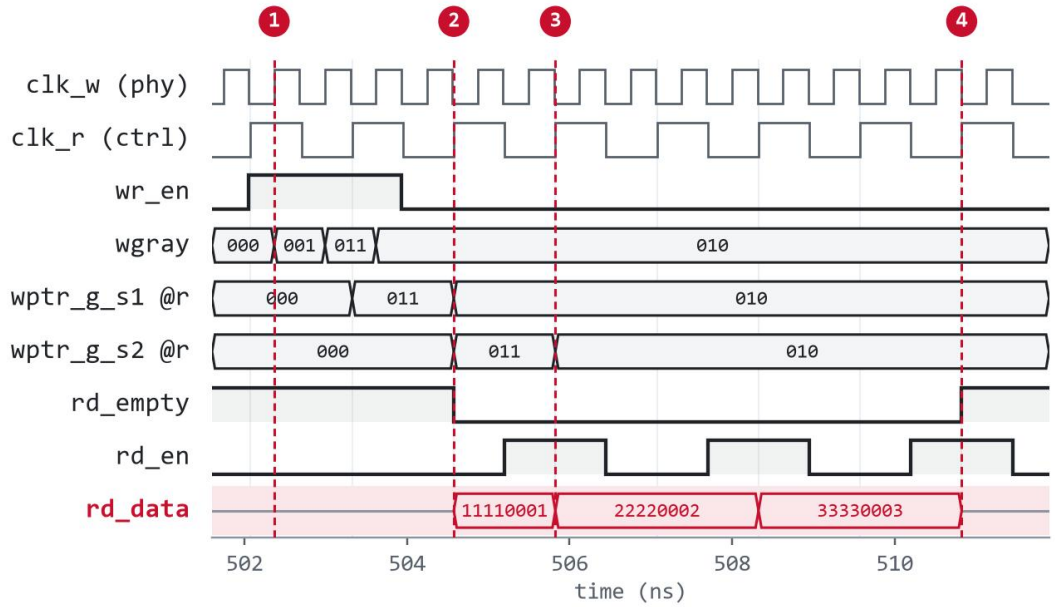
tb_hsl_blocks의 독립 인스턴스 (E7 · T6) — 온칩 gearbox · FIFO는 입력이 0이라 이 블록 TB가 동작 근거

E7 hsl_gearbox · 32b 워드 → 16b beat × 2



1. 495.6 ns din_valid 수락 → 워드 hold, req_tog 토글, din_ready 0
2. 496.7 ns PHY 클럭 2-FF 뒤 req_s2 반영
3. 497.4 ns 하위 C3D4 → A1B2 두 beat, 이어서 ack_tog 토글
4. 499.4 ns ctrl 쪽 ack_s2 → din_ready 1, 다음 워드 수락 가능

T6 hsl_cdc_fifo · 쓰기 clk_phy → 읽기 clk



1. 502.4 ns wgray 000 → 001 → 011 → 010, 매번 1비트만 변화
2. 504.6 ns wptr_g_s2 도착 → rd_empty 0. s1이 001을 건너뛰어도 유효한 값
3. 505.8 ns부터 pop: 11110001 → 22220002 → 33330003
4. 510.8 ns 세 번째 pop 뒤 rd_empty 1

디버그 기록: 문서화된 이슈 6건 + 이번 재검증에서 찾은 5건

왼쪽은 docs/에 원인과 조치가 적힌 항목, 오른쪽은 AI 에이전트 보조 재실행 · 합성 · 코드 리딩에서 새로 본 항목 (제안만, 저장소 수정 없음)

저장소에 기록된 이슈 → 원인 → 조치

이슈	원인	조치 · 근거
E3 · cmd_ready 기대값	스텁 큐가 push_ready를 0에 묶어 MISSION에서도 ready 0	큐 구현 후 기대값을 '리셋 직후 · full일 때 0'으로 바꾸고 이유를 TB 주석에 (e0_e1_notes)
E12 · PHY 주기 1248 ps	반주기를 정수 312 ps로 쓰면 4구간 합이 1248 ps ≠ 1250	#312.5ps = PHY_PERIOD_PS/2, 문서 · pkg · TB · SDC 대조표 (e12_clocks)
E1 · soft reset 자기 해제	CSR까지 soft reset에 넣으면 CTRL bit 0이 다음 클럭에 0	CSR은 rst_sync_n만, 나머지는 rst_logic_n (05_pd_guide)
E8 · CDC 포인터 제약	false path는 그 경로의 지연을 STA가 보지 않게 만듦	목적지 주기 set_max_delay 1.25 / 0.625 ns, false path 금지
E11 · PRBS · ERRCNT	생성기 0끼리 비교하면 고장 통과, 16b 랩이면 0으로 돌아와 정상처럼 보임	seed 0 무시 · 리셋 시드 5A, 카운터 FFFF 포화 (06_dft_plan)
E12 · I/O delay 4.0 ns	v0.1의 10 ns 클럭 기준 40%가 그대로 남음	800 MHz 주기의 40% = 0.5 ns로 수정

이번 재검증에서 새로 찾은 것

A 타임스케일 반올림: #312.5ps → 313 ps

1 ps 정밀도라 u_phy_clk 구간이 313 · 313 · 313 · 311 ps, 블록 TB clk_phy는 626 ps (VCD 측정). 제안: timeprecision 100fs

B TB 리셋 해제 레이스

rst_n을 posedge 직후 해제 → 같은 엣지에서 sync_r[0] = 1, rst_sync_n은 2클럭이 아닌 1클럭 뒤 (5.625 → 6.875 ns). 제안: negedge 해제

C SDC 대상이 합성 후 사라짐

gear · deskew · FIFO 출력은 unused_path로만 가서 Yosys가 제거, 이어서 u_phy_clk도 제거 → 생성 클럭과 FIFO max_delay의 대상이 없음

D 미지원 opcode가 큐를 멈춤 (코드 리딩)

REF · 예약 opcode가 head에 오면 head_ok = 0 → 영구 정지. 시뮬 안 함 (scheduler 51행 미도달). 제안: push에서 거절

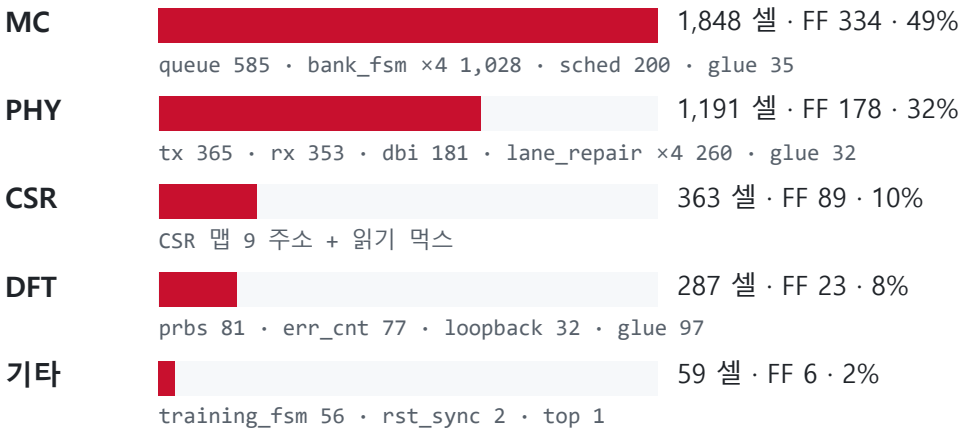
E 리셋 없는 플롭 200개

큐 mem_* 4 × 50b가 리셋 분기에 없음 → $_DFFE_PP_ 200. 출력은 empty 마스크로 안전, DFT 규칙 '리셋 없는 플롭 금지'와 충돌

PPA: 합성 셀로 본 면적 분포와 SDC가 고정한 타이밍 의도

Yosys 0.52 generic 셀은 면적 proxy일 뿐 — 셀 라이브러리 · μm² · mW · slack은 저장소에 없고, 여기서도 만들지 않았다

블록별 generic 셀 · FF (계층 합성, 인스턴스 수 반영)



합계 3,748 셀 · FF 630 (계층) · 평탄화 3,492 셀 · FF 630 (`$scopeinfo` 25 제외, 칩 단위 상수 전파 후)

온칩에서 제거된 clk_phy 블록의 단독 합성: gearbox 96 / FF 72 · cdc_fifo 378 / 146 · deskew 126 / 36

합성 준비는 스크래치 복사본에서만: `import hsl_pkg::*` → localparam 붙여넣기 (Yosys 미지원), `hsl_phy_clk` 블랙박스, `hsl_dbi`의 `always_comb` + 모듈 스코프 `integer bi` → latch 판정이라 `always @*`

syn/constraints/hsl_base_die_lite.sdc L8, L12–15, L22, L56–62

```
 8  create_clock -name clk -period $ctrl_period [get_ports clk]   (1)

12  create_generated_clock -name clk_phy \                        (2)
13    -source [get_ports clk] \
14    -multiply_by 2 \
15    [get_pins u_phy/u_phy_clk/clk_phy]

22  set_false_path -from [get_ports rst_n]                        (3)

56  set_max_delay $ctrl_period \                                  (4)
57    -from [get_cells u_phy/u_fifo/wptr_g_s1*] \
58    -to   [get_cells u_phy/u_fifo/wptr_g_s2*]
59
60  set_max_delay $phy_period \
61    -from [get_cells u_phy/u_fifo/rptr_g_s1*] \
62    -to   [get_cells u_phy/u_fifo/rptr_g_s2*]
```

1. `clk` 1.25 ns = 800 MHz, E12 대조표와 동일
2. `clk_phy` = clk × 2, PLL 자리의 생성 클럭
3. `rst_n` false path는 v0.1 단순화 — recovery/removal 확인 필요
4. 포인터 동기화는 false path 대신 목적지 주기 `set_max_delay`

성능 (T4 파형 측정)

ACT → WR 15클럭 · RD → 응답 6클럭. bypass 프레이머는 ctrl 클럭당 1 beat (WR 5클럭) — 목표 대역 경로 아님

전력 레버 (mW 미측정)

DBI-DC로 DQ+DBI 9핀 중 high ≤ 4 · CKE는 RESET에서만 0 · 버스트 밖 `pin_dq_oe` = 0

PD 가이드 (docs/05_pd_guide.md)

deskew 4단 = 8 UI 허용 스큐 · SRAM 매크로 없음 · PHY 핀 로직은 범프 쪽, CSR · FSM은 안쪽

AI 활용: 생성은 보조, 판정은 시뮬레이터 — 이 덱도 같은 원칙으로 자동화

저장소 문서의 원칙, 실무에서 쓴 방식, 이 포트폴리오를 만든 파이프라인을 구분해 적는다

저장소에 적힌 AI 사용 원칙

> "직무 우대의 LLM/EDA 항목은 별도 툴로 만들지 않는다. E2 이후 디렉티드 테스트를 추가할 때, 테스트 스케치와 SDC 문장을 리뷰받는 용도로만 쓴다. 생성 결과를 시뮬레이터와 파형으로 확인하는 쪽이 포트폴리오다."
>
> — docs/00_study_map.md

적용 방식

- AI 제안은 테스트 스케치 · SDC 문장 리뷰로 한정, 스펙 숫자는 문서가 먼저
- 채택 기준은 시뮬레이터: 이름 있는 PASS/FAIL과 VCD 파형으로만 승인
- 이 덱: 실행 → 측정 → 그림을 스크립트로 재현, 수치를 손으로 옮기지 않음

실제 사용 — 실무와 이 포트폴리오

- **실무 (SoC 디지털 설계)** Claude Opus · GPT로 I2C Master IP RTL의 FSM · 레지스터 뼈대와 Chip Top UVM 환경의 Driver · Monitor · Sequence 코드를 생성 → 설계 초안 · 검증 셋업 시간 2배 이상 단축
- **이 포트폴리오** 재실행 · VCD 파형 분석 · 합성 리포트 · 덱 생성은 AI 에이전트가 수행, 작업 지시와 결과 확인은 직접

이 포트폴리오를 만든 파이프라인 (Cursor AI 에이전트 + 스크립트)

1	**저장소 고정**	commit `4feb39f` · RTL/TB 수정 없음 · git status로 확인
2	**Verilator 5.052 실행**	회귀 13/13 · `--trace` VCD · `--coverage` → regress.log · full.vcd · coverage.dat
3	**VCD 파서 → 파형**	직접 쓴 VCD 파서 + matplotlib · 마커 시각은 VCD 값에서 · 파형 8장
4	**코드 캡처**	pygments · 원본 줄 번호 유지 · 슬라이드에서 9 pt 이상 · 11장
5	**Yosys 0.52 합성**	generic `synth` 계층 / 평탄화 → 블록별 셀 · FF 집계
6	**덱 생성 · 리뷰**	python-pptx로 편집 가능한 도형 · PowerPoint로 전 슬라이드 렌더 → 겹침 · 넘침 수정

19쪽 발견 A–E는 에이전트 보조 분석의 제안이며 RTL/TB는 수정하지 않았다

11 마무리

JD 연결: 스펙 → 아키텍처 → RTL → 검증 → PD 가이드를 한 슬라이스에서

SK hynix HBM Digital Design (RTL Design) — 담당 업무 · 자격 요건 · 우대 사항과 이 프로젝트의 근거

JD 항목	이 프로젝트의 근거	쪽
요구 구체화 · 상세 스펙	기능 7개 · 파라미터 · 수락 조건을 문서에 먼저 고정, RTL에만 있는 숫자 금지	3
PPA 평가 → Architecture	A/B/C를 성능 · 전력 · 면적 · 리스크로 비교, A → B 이행과 E12 대조표	4
RTL 설계 · Function 검증	21 모듈 1,462줄 · 회귀 13/13 PASS · 라인 87.5% / 분기 96.6%	5–13
PD Guide · Floorplan · Integration	클럭 · 리셋 · CDC 제약, deskew 8 UI, 매크로 없음, 블록별 셀 · FF	20
Wafer Test Flow · 이슈 분석	ID → PRBS 루프백 → REMAP → 트레이닝 순서, 재검증 발견 5건	12, 17, 19
High Speed Digital PHY / IP	프레이머 · DBI-DC · 레인 리페어 · 토글 기어박스 · Gray FIFO · deskew	7, 10–11, 17–18
DRAM Memory Controller	깊이 4 큐 · in-order 스케줄러 · 뱅크 FSM ×4, 타이밍 소유 분리	6, 9, 16
DRAM DFT / DFD	PRBS7 · 병렬 루프백 · 16b 포화 ERRCNT · STATUS · REMAP 관측점	7, 11, 17
Full-Custom + Digital 경계	핀 리스트로 경계 고정: 디지털은 값 · enable, 아날로그는 드라이버 · 샘플러	2, 5
LLM / AI 기반 EDA 자동화	실무: LLM으로 RTL 뼈대 · UVM 코드 생성, 초안 · 셋업 시간 2배 이상 단축 이 덱: AI 에이전트로 재실행 · 파형 · 커버리지 · 합성 집계 자동화	21

다음 단계

- REF 발행 · tRFC, 미지원 opcode 거절 (D)
- DBI 읽기 · 타이밍 재프로그램 · ERRCNT clear 테스트로 커버리지 구멍 닫기
- 랜덤 트래픽 + 스코어보드로 토글 56.6% 개선
- 실제 셀 라이브러리로 STA · 면적 · 전력 숫자 확보
- OoO 스케줄러 — 뱅크 FSM 분리 구조 재사용

한 줄 요약

스펙의 숫자를 문서 → `hsl_pkg` → RTL → TB → SDC까지 같은 값으로 묶고, 그 일치를 시뮬레이터 · 파형 · 합성 결과로 보여 준다.

용현 권

kyongh0331@naver.com

github.com/yhkwon-git/High_Speed_Link_Lite